LAB1 – Array and OOP Warm Up

Objective: In this lab, you going to work on a small project that mainly focuses on reference type one dimensional array and its operations. After the completion of this lab:

1. You should be able to confidently use array as your fundamental data structure and able to perform required operations.
2. You should also be able to confidently handle reference type array and its operation, such as insert, delete or search for an element while the elements are reference type
3. You should be able to provide a testDriver to test your implementation
4. Able to access private variable with getter/setter methods

Create an application called ***Restaurant*** that has the following classes:
A ***Customer*** class that minimally stores the following data fields for a customer:

- name:String
- phone:String
- seat:int //how many seats needed
- highchair:boolean //if highchair is needed, true for yes and false for no

This class is half done and provided in the folder, please complete this class by providing the following methods. Do not change the provided implementation without consulting with your instructor.

1. Override the toString method nherited from Object class. Print out the customer information in the way you want.
2. Override the equals method inherited from Object class. Return true if two customer's name and phone number are the same. False otherwise.
3. Implement Comparable interface, the comparison is based on *seat*. If a customer has a larger *seat* than another customer, it will be considered as a larger customer.

The restaurant is going to trace its current customer and waited customer using two arrays. When a group of people checked in the restaurant, they only keep one customer information. So, one customer can take more than one seat based on his/her need. To avoid long waiting, the waited line will accept no more than 10 waiting group.
A ***Restaurant*** class that minimally stores the following information:

- maximum seat
- A one-dimensional Customer array stores the current customer information (use the basic array instead of ArrayList) //referred as currList
- Number of customers in currList

- A one-dimensional Customer array used as waitlist stored customer who would like to wait (use the basic array instead of ArrayList) //referred as waitLine
- Number of customers in waitList

The following methods should be provided:

- A constructor that initializes all data fields.
  - Initialize maximum seat with a passed parameter
  - Initialize currList with maximum seat
  - Initialize waitList with size 10
- A Method displays the first n elements either on the currList or waitList
  - This method accepts two parameters, one is a customer array, and the other is an int value
  - If the number of elements in the passed list is less than n, stop when all elements are printed
- A method to check how many seats is already taken
  - This method will return the number of seats used by customer. This information can be obtained by sum seat information of all customers in the currList
- A search method accepts a customer as parameter and return an int
  - Return 0 if the customer is found in currList
  - Return 1 is the customer is found in waitList
  - Return -1 otherwise.
- An add method accept a customer as parameter
  - If the remaining seat is more than what the customer requires and the new customer is not in the currList or waitList, add this customer to the end of currList. Print message to show the result
  - Else if the waitList contains less than 10 customers, add this new customer into the waitList. Print message to show the result
- A remove method accepts a customer as parameter
  - If the customer is found in the waitList, remove this customer from waitList
  - If the customer is found in the currList, remove this customer from the currList. After that, If the waitList is not empty, move the first element in waitList whose seat is less or equal to the customer's seat to currList to fill the spot.

There is a **testDriver** class provided. Modify it based on your need to test your program, you are required to test all the implemented methods in the Restaurant class.

What to submit:
Customer.java, Restaurant.java and testDriver.java.

Comment:
If your version of Eclipse asks if you want to create module or not during the process of new a java project, click "Don't create."